Fault-Tolerant Control of a Quadrotor After Complete Rotor Loss: Residual Koopman MPC and Certified Fault-Detection-Delay Budgets

EE650 Basics of Modern Control Systems — Progress Report 1

[Team member names]

October 2026

Abstract

When one rotor of a quadrotor fails completely, the vehicle can no longer hover level. Its only steady state is a constant spin about a tilted body axis, the *relaxed hover* of Mueller and D'Andrea. This project studies two questions about this regime, in simulation. The first (RQ1, from our proposal) asks whether a Koopman (EDMDc) model that learns only the mismatch of the known physics model predicts and controls better, inside model predictive control, than the physics model alone or a full-state Koopman model. The second (RQ2, added after the proposal) asks how long fault detection may take before recovery into the relaxed hover is no longer guaranteed. This report covers the first phase of the work. We built a simulator of the vehicle used by Mueller and D'Andrea and validated it against three published papers. It reproduces their two-rotor and three-rotor-loss equilibria to every published digit and five hover solutions of their detailed model to within 0.7%. We analysed controllability and observability and designed LQR baselines. For RQ1, the residual model gives the lowest multi-step prediction error on the training plant and needs less data, and a residual-corrected MPC removes about one third of a steady altitude error. The learned models do not yet generalize to an unseen plant. For RQ2, the published recovery strategy tolerates only about 10 ms of detection delay in our simulation, while reported detectors need 30–130 ms. A first saturation-aware LMI certificate covers 22% of the way from the equilibrium to the state right after a failure in hover. We close with the limitations and the plan for the remaining weeks.

Contents

1 Introduction **3**

2 Related work **3**

3 System model **4**
3.1 Equations of motion 4
3.2 Parameters and truth plants 5

4 Model validation **5**

5 Linear analysis and baseline control **6**
5.1 Linear models 6
5.2 Controllability and observability 6
5.3 LQR baselines 6

6 RQ1: Residual Koopman models and MPC **7**
6.1 Predictors . . . 7
6.2 Prediction results . . . 7
6.3 MPC in closed loop . . . 8

7 RQ2: Fault-detection delay **10**
7.1 The undetected phase . . . 10
7.2 Effect of detection delay . . . 10
7.3 A first saturation-aware certificate . . . 11

8 Limitations **12**

9 Plan for the remaining weeks **12**

10 Conclusion **12**

A Reproducibility **13**

1 Introduction

Quadrotors have no actuator redundancy. If one rotor stops, the remaining three cannot produce zero yaw torque while carrying the weight, so the vehicle starts to spin. Mueller and D'Andrea showed that the vehicle can still be controlled by giving up the yaw angle. The vehicle spins at a constant rate about a body-fixed axis, and it translates by tilting this axis [1, 2]. In body-fixed coordinates this *relaxed hover* is a fixed point, so linear time-invariant tools apply. They stabilised it with LQR and demonstrated it in flight.

Later work improved the controllers. It includes robust and LPV designs [3, 4], incremental nonlinear dynamic inversion [5], recovery from large initial upsets [6] and nonlinear MPC [7]. Learning-based models have also entered quadrotor control. Koopman-operator methods give linear predictors that fit convex MPC [8, 9], and residual Koopman models correct a nominal model with a learned term [10–12].

Research questions.

- **RQ1 (proposal).** Can a Koopman/EDMDc residual model, built around the analytically known spinning-equilibrium dynamics of a completely failed quadrotor, provide a better long-horizon predictive model for MPC than either the analytical failure model alone or a generic full-state Koopman model?
- **RQ2 (added).** For a quadrotor that loses one rotor completely, what is the largest fault-detection delay $\tau^\star$ for which recovery into the relaxed hover is guaranteed, given the healthy controller, the failure-mode controller, thrust saturation and motor lag? How do the controllers of RQ1 change $\tau^\star$?

RQ2 was added because two gaps appeared while we reviewed the literature. First, a learned predictor says nothing about *where* recovery is guaranteed. Second, every recovery method with a guarantee that we found assumes that the failure is detected at the instant it happens [13, 14]. The only published tolerance figure we found is empirical and for a different vehicle [15]. The two questions are linked: each controller that captures the vehicle after detection can be compared by the delay it tolerates.

Work reported here. (i) A simulator of the Mueller–D'Andrea vehicle, validated against three papers (Section 4). (ii) Linear analysis and LQR baselines (Section 5). (iii) First results for RQ1: three predictors and two MPC controllers (Section 6). (iv) First results for RQ2: the effect of detection delay, and a first LMI certificate (Section 7). Section 8 lists the limitations and Section 9 the remaining work. Everything is in simulation; no hardware is used.

2 Related work

Complete rotor loss. Freddi, Lanzon and Longhi gave up yaw control and stabilised a horizontal spin with feedback linearization and H_∞ loop shaping [3, 16]. Mueller and D'Andrea derived the relaxed hover for one, two and three lost propellers, proved controllability of its linearization and validated it in flight [1, 2]. Stephan et al. designed an LPV controller scheduled on the yaw rate to cover the transient after the loss [4]. Sun et al. used incremental nonlinear dynamic inversion at high speed [5] and a quadratic-programming allocation for recovery from large upsets [6]. Nan et al. used nonlinear MPC with per-rotor thrust limits [7]. The upset-recovery study names fault-detection delay and actuator saturation as reasons why linear controllers may fail. It does not quantify where they fail [6].

Guarantees and detection delay. Simha et al. give an analytic region of attraction for a geometric controller, with the fault assumed to be known from the start [13]. Al Lawati and Akhtar prove path invariance under quasi-static feedback and state that they assume a reliable fault-detection module and an instantaneous transition [14]. For a variable-pitch vehicle, a maximum tolerable detection delay of 180 ms was found by simulation sweeps [15]. Reported detectors for rotor failure need roughly 30–130 ms [17]. Set-theoretic fault-tolerant control uses invariant and reachable sets to reason about detection and reconfiguration in general [18], and detection delay has been analysed for general fault-tolerant systems [19]. A motion planner based on backward reachability exists for the failed quadrotor after the fault is handled [20].

Koopman models and residual learning. EDMD approximates the Koopman operator on a dictionary of observables [9]. With inputs it gives linear predictors for MPC [8, 21]. Koopman MPC exists for healthy quadrotors [22, 23]. Koopman-based fault-tolerant control and fault detection address partial loss of effectiveness, with all rotors still working [24, 25]. Residual Koopman models correct a nominal model for soft robots, ground vehicles, quadrupeds and healthy quadrotors [10–12, 26]. Todde et al. evaluate the residual outside the quadratic program and inject it as a measured disturbance, which keeps the problem convex. They list formal robustness guarantees as future work [10]. We did not find residual Koopman models applied to the complete-loss, spinning-equilibrium regime, nor a certified bound on the tolerable detection delay. We state this as the outcome of our search, not as proof that no such work exists.

3 System model

3.1 Equations of motion

We follow the model of [1]. The body frame has z along the thrust direction. Rotor 1 is on the $+x$ arm, rotor 2 on $+y$, rotor 3 on $-x$ and rotor 4 on $-y$, all at distance l. Rotors 1 and 3 spin with negative speed, and rotors 2 and 4 with positive speed. With thrusts $f_i = \kappa_f \omega_i^2$, position $\boldsymbol{d}$, attitude R and body rates $\boldsymbol{\omega} = (p, q, r)$,

$$m\ddot{\boldsymbol{d}} = R\,\boldsymbol{e}_3 \textstyle\sum_i f_i - mg\,\boldsymbol{e}_3, \tag{1}$$

$$I^B\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (I^T\boldsymbol{\omega} + I^P_{zz}\,\omega_\Sigma\,\boldsymbol{e}_3), \qquad \omega_\Sigma = \textstyle\sum_i \omega_i, \tag{2}$$

$$\boldsymbol{\tau} = \big(\, l(f_2 - f_4),\; l(f_3 - f_1),\; \kappa_\tau(f_1 - f_2 + f_3 - f_4) - \gamma r \,\big).$$

Here I^B is the body inertia, $I^T = I^B + 4I^P$ includes the propellers, and γ is a linear yaw-drag coefficient. As in the paper, the term $I^P\dot{\boldsymbol{\omega}}$ is neglected. Each motor follows its command with a first-order lag, $\dot{f}_i = (\mathrm{sat}(u_i) - f_i)/\sigma$. A running motor produces $f_i \in [f_{\min}, f_{\max}]$, and a command $u_i \le 0$ stops it. A failed rotor produces no thrust. The attitude is stored as a quaternion and integrated with RK4 at 0.5–1 ms. Controllers run at 1–5 ms with a zero-order hold.

Reduced attitude. After the failure we control only the direction $\boldsymbol{n} = R^\top \boldsymbol{e}_3$, which is the inertial "up" direction seen from the body. It obeys

$$\dot{\boldsymbol{n}} = -\boldsymbol{\omega} \times \boldsymbol{n}. \tag{3}$$

Relaxed hover. An equilibrium has constant $\boldsymbol{\omega}$ and $\boldsymbol{n}$ in the body frame. This requires $\dot{\boldsymbol{\omega}} = 0$ in (2), $\boldsymbol{n} \parallel \boldsymbol{\omega}$ (so that $\dot{\boldsymbol{n}} = 0$), and $f_\Sigma n_z = mg$ for zero mean vertical acceleration. With rotor 4 lost, $f_1 = f_3$ and $f_2 = \rho f_1$, the rate equations have a closed form: $p = 0$, $r = \kappa_\tau(f_1 - f_2 + f_3)/\gamma$ and $q = l f_2/((I^T_{zz} - I^T_{xx})r + I^P_{zz}\omega_\Sigma)$. A one-dimensional root search on f_1 then gives the equilibrium.

3.2 Parameters and truth plants

All parameters are those of the vehicle in [1, Sec. V-A] (Table 1). For the learning experiments we also use *truth plants* that contain effects the nominal model lacks. The first is a loss of thrust efficiency at high body rate, $\eta = 1 - k\|\boldsymbol{\omega}\|^2$; the paper reports less thrust than expected while spinning. The others are an inertia error and a different motor time constant. Family A is used for training. Family B uses different magnitudes and is held out.

Table 1: Vehicle parameters [1] and truth-plant families.

Quantity	Symbol	Value
Mass, arm length	m, l	$0.50\,\mathrm{kg}$, $0.17\,\mathrm{m}$
Total inertia	I^T_{xx}, I^T_{zz}	3.2, $5.5 \times 10^{-3}\,\mathrm{kg\,m^2}$
Propeller inertia	I^P_{zz}	$1.5 \times 10^{-5}\,\mathrm{kg\,m^2}$
Thrust, torque coefficients	κ_f, κ_τ	$6.41 \times 10^{-6}\,\mathrm{N\,s^2}$, $0.0169\,\mathrm{m}$
Yaw drag	γ	$2.75 \times 10^{-3}\,\mathrm{N\,m\,s}$
Thrust range, motor lag	$[f_{\min}, f_{\max}]$, σ	$[0.2, 3.8]\,\mathrm{N}$, $15\,\mathrm{ms}$
Truth family A		$k = 1.5 \times 10^{-4}$, $I_{xx} \times 1.08$, $I_{zz} \times 0.95$, $\sigma = 20\,\mathrm{ms}$
Truth family B (held out)		$k = 2.5 \times 10^{-4}$, $I_{xx} \times 0.94$, $I_{zz} \times 1.06$, $\sigma = 12\,\mathrm{ms}$

4 Model validation

We validated the simulator in two ways. First, 45 automated tests compare it with hand-derived results: free fall; thrust direction; closed-form torque-free precession; conservation of inertial angular momentum and kinetic energy (to 10^{-8}); reduced-attitude kinematics against the matrix exponential; the first-order yaw spin-up; the motor-lag step response; the fourth-order convergence of RK4; and an independent transcription of the paper's rate equations. Second, we compared it with published numbers from three papers (Table 2).

Table 2: Comparison with published results. Full tables: `results/tables/validation_report.md`.

Source	What was compared	Result
ICRA 2014 [1]	Two-rotor-loss equilibrium, eqs. (57)–(59)	$f = 2.453\,\mathrm{N}$, $\omega_1 = -618.6\,\mathrm{rad/s}$, $r = 30.14\,\mathrm{rad/s}$ vs. 2.45, −619, 30.1: all digits ($\leq$0.14%)
ICRA 2014	Three-rotor-loss equilibrium, eqs. (62)–(64)	$\boldsymbol{\omega} = (14.68, 0, 32.99)$ vs. (14.7, 0, 33.0), $\boldsymbol{n} = (0.407, 0, 0.914)$ vs. (0.41, 0, 0.91): all digits
ICRA 2014	One-rotor-loss equilibrium, $\rho = 0.5$	f, r, n_z within 0.5%; q and n_y 6% lower (see text)
ICRA 2014	Coupling constant $\bar{a}$, eq. (28); controllability loss, eq. (39)	Exact; controllability rank drops from 7 to 6 with p uncontrollable exactly when $\bar{a} = 0$
IJRR 2016 [2]	Five hover solutions (no failure; one; two opposing; two adjacent; three rotors lost), independent implementation of their model	0–4% with the published parameters; within 0.7% with $I_{xx} = 2.75 \times 10^{-3}$, inside the rounding of the published 2.7×10^{-3}
Sun et al. [5]	Bebop 2, two-rotor loss, eq. (60)	Equal to the paper's formula; the paper's quoted values are 1.5% below its own formula
Flight data [1, 2]	Time to spin up to $10\,\mathrm{rad/s}$ on two rotors	Model $0.80\,\mathrm{s}$ vs. flight $0.62\,\mathrm{s}$: **model about 30% slower**

Two points need comment. In the one-rotor case, the paper's published values do not satisfy its own roll-rate equation: they leave a residual torque of $-0.011\,\mathrm{N\,m}$. Its other two equilibria match exactly with the same parameters, so we treat this as an inconsistency in that one published case. The IJRR model computes thrust from the propeller speed relative to the air, $\Omega_i + r$, and uses quadratic drag. We implemented it separately. With these effects switched off, it reduces exactly to our simulator, which the tests confirm. The remaining gap is with flight data: the real vehicle spun up faster than either model predicts, and the IJRR authors report a measured spin rate 20% below their simulation. The results of Section 7 depend on spin-up speed and must be read with this in mind.

5 Linear analysis and baseline control

5.1 Linear models

At hover the healthy vehicle has 12 states and four inputs. After the failure we use $\boldsymbol{s} = (p, q, r, n_x, n_y, z, v_z)$ with inputs (f_1, f_2, f_3). These equations are autonomous in body coordinates, so their linearization about the relaxed hover is time invariant. Both models are Jacobians of the exact equations, checked against finite differences. The failed model has open-loop eigenvalues $\pm 10.10j$ (gyroscopic coupling $\bar{a}$), $\pm 19.56j$ (equal to $\|\bar{\boldsymbol{\omega}}\|$), -0.51 (yaw drag) and a double integrator (altitude). The exact coefficient of n_y in $\dot{n}_x$ is $\bar{r} + \bar{q}\bar{n}_y/\bar{n}_z = 20.3$; the simplified matrix in [1] uses $\bar{r} = 18.8$.

5.2 Controllability and observability

Both models are controllable: the controllability matrix has full rank and every mode passes the PBH test. Table 3 lists observability for four sensor sets of the failed vehicle. Gyros alone see neither tilt nor altitude. The relaxed hover is tilted ($\bar{n}_y \neq 0$), so the vertical acceleration depends on tilt to first order. Gyro plus barometer therefore makes the full state observable. The smallest eigenvalue of the 1 s observability Gramian is, however, about 2,700 times smaller than with a direct tilt measurement. This is a structural property, not a practical estimator. For the healthy vehicle, gyro plus position leaves the heading unobservable, as expected.

Table 3: Observability of the failed vehicle's linear model (7 states).

Sensors	Rank	Unobservable	$\lambda_{\min}$ of $W_o(1\,\mathrm{s})$
Gyro (p, q, r)	3/7	n_x, n_y, z, v_z	0
Gyro + tilt estimate (n_x, n_y)	5/7	z, v_z	0
Gyro + barometer (z)	7/7	none	2.4×10^{-5}
Gyro + tilt + barometer	7/7	none	6.6×10^{-2}

5.3 LQR baselines

The healthy LQR uses Bryson weights. The failure-mode LQR uses the weights of [1] (1 on p, q; 20 on n_x, n_y; $1\,\mathrm{N}^{-2}$ on the inputs). We add the motor states and altitude, and reconstruct the motor states from the controller's own commands. Table 4 gives the closed-loop properties. On the nonlinear model with motor lag and thrust limits, the failure LQR returns from a perturbed start: tilt and altitude settle in 1–2 s and the spin rate in about 4 s (Fig. 1). The input-delay margin of the failure LQR is only 18 ms, against 40 ms for the healthy one.

Table 4: LQR designs. The delay margin is the largest pure input delay that keeps the linear closed loop stable.

Controller	Slowest closed-loop poles	Delay margin
Healthy LQR (12 states)	$-1.72 \pm 1.02j$, -1.89, $-1.94 \pm 1.56j$	40 ms
Failure LQR (7 + 3 motor states)	-1.09, $-2.01 \pm 1.04j$, $-3.80 \pm 19.68j$	18 ms

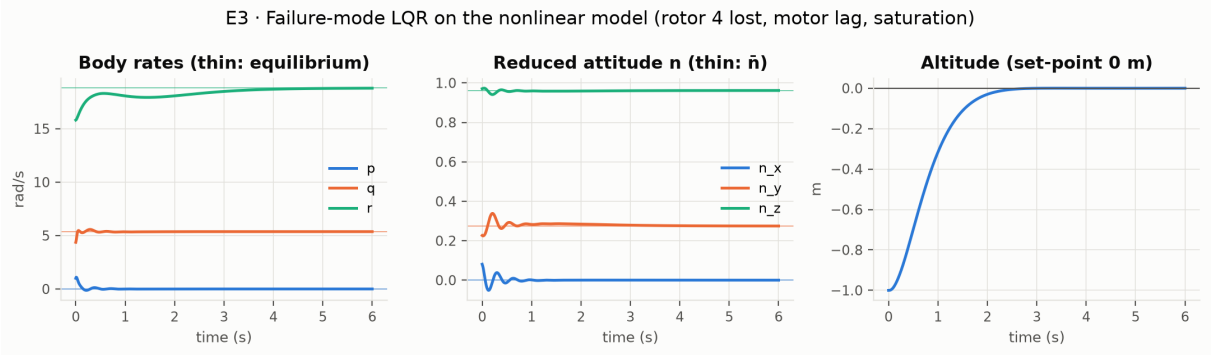


Figure 1: Failure-mode LQR on the nonlinear model (rotor 4 lost, motor lag, thrust limits), starting from a perturbed relaxed hover 1 m below the set-point.

6 RQ1: Residual Koopman models and MPC

6.1 Predictors

All predictors use the same data. These are 40 closed-loop episodes of 4 s on truth family A, sampled at $T_s = 5$ ms, with the failure LQR and random excitation added to the commands. The measured state is $\boldsymbol{x} = (p, q, r, n_x, n_y, z, v_z)$ (deviations from the relaxed hover). The thrusts the controller knows are the nominal first-order filter $\hat{\boldsymbol{f}}$ of its own commands.

- **Analytical.** The Mueller–D'Andrea linear model with motor states, discretized with a zero-order hold: $\boldsymbol{s}_{k+1} = A_d \boldsymbol{s}_k + B_d \boldsymbol{u}_k$, with $\boldsymbol{s} = (\boldsymbol{x}, \hat{\boldsymbol{f}})$.
- **Full-state EDMDc** [8, 9]. Lifted state $\boldsymbol{z} = \psi(\boldsymbol{\xi})$ with $\boldsymbol{\xi} = (p, q, r, n_x, n_y, v_z, \hat{\boldsymbol{f}})$. The dictionary ψ holds a constant, the standardized $\boldsymbol{\xi}$ and all its degree-2 monomials (55 functions). We fit $\boldsymbol{z}_{k+1} \approx A\boldsymbol{z}_k + B\boldsymbol{u}_k$ by ridge-regularized least squares.
- **Residual EDMDc (proposed).** The analytical prediction plus a learned one-step correction,

$$\boldsymbol{x}_{k+1} = [A_d \boldsymbol{s}_k + B_d \boldsymbol{u}_k]_{1:7} + \mathrm{clip}\left(W\psi(\boldsymbol{\xi}_k, \boldsymbol{u}_k),\, \pm \boldsymbol{d}_{\max}\right), \tag{4}$$

  where W is fitted by ridge regression on the one-step residuals and $\boldsymbol{d}_{\max}$ is the 99.5th percentile of the training residuals.

6.2 Prediction results

Figure 2 and Table 5 show the open-loop multi-step error on held-out episodes. On family A the residual model is the most accurate at every horizon of 50 ms and above. The analytical model drifts: its undamped modes and the unmodelled thrust loss make it worse than the trivial "no change" predictor. Full-state EDMDc has 4 of its 55 eigenvalues outside the unit circle (spectral radius 1.0054). It falls behind the no-change predictor between 2 and 3 s, as the proposal predicted. At 3 s the residual model also exceeds the no-change error, so long open-loop horizons need feedback.

Table 5: Normalized multi-step RMSE (mean over the 7 states), models trained on family A.

	Test family A			Test family B (held out)		
Predictor	0.1 s	1 s	3 s	0.1 s	1 s	3 s
Analytical	2.29	5.13	13.6	3.07	5.57	9.86
Full-state EDMDc	0.13	0.70	4.97	4.37	12.3	148
Residual EDMDc	**0.11**	**0.42**	3.55	4.39	11.8	8.66
No change	1.93	2.03	2.22	1.95	1.93	2.03

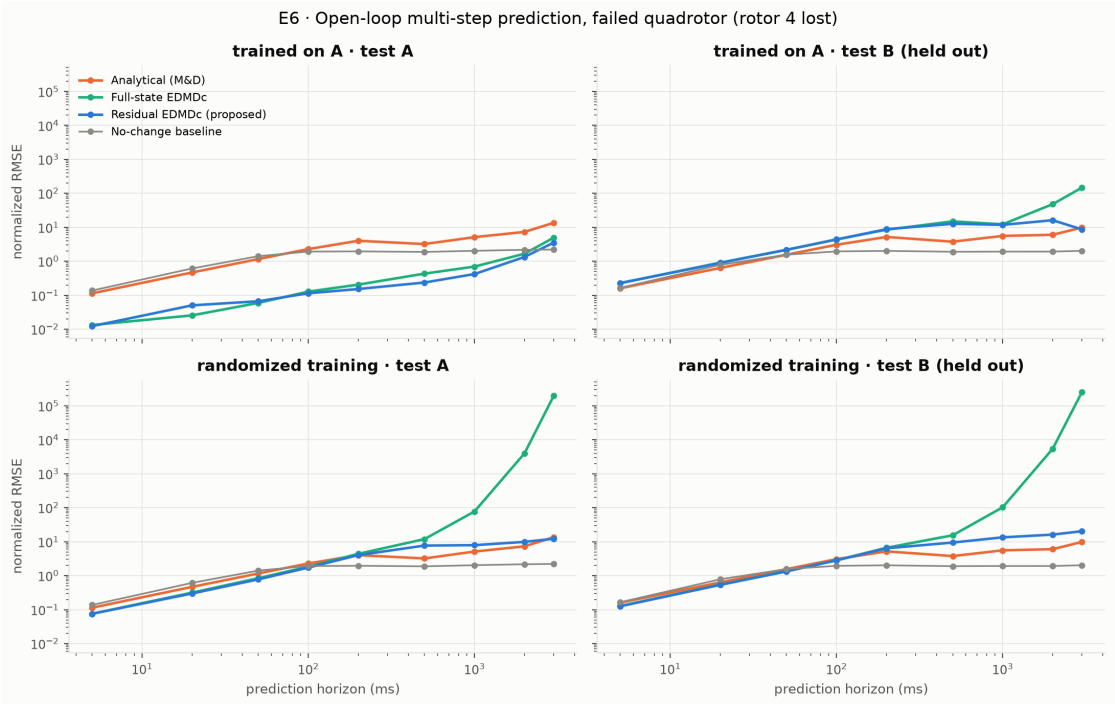


Figure 2: Multi-step prediction error. Top: models trained on family A. Bottom: models trained on randomized plants. Left: test on family A; right: test on the held-out family B.

Data efficiency. At a 0.5 s horizon, the residual model trained on 10 episodes (error 0.32) is better than the full-state model trained on 40 (0.43) (Fig. 3). This matters because data from failed flights is scarce.

Generalization. On the held-out family B both learned models are worse than the analytical model (Table 5). Training on randomized plants, one random plant per episode, made both models worse on family A as well (Fig. 2, bottom). The likely reason is that the motor time constant and thrust loss of the current plant cannot be inferred from a single state sample, so a static regression averages over them. This is a real limitation of the current approach and motivates online adaptation of W, which is linear in the data.

6.3 MPC in closed loop

Both MPCs use the analytical discrete model, a horizon of $N = 20$ steps of 5 ms, a terminal cost from the discrete Riccati equation, and thrust limits. They solve

$$\begin{aligned} \min_{\boldsymbol{u}_0,\dots,\boldsymbol{u}_{N-1}} \quad & \textstyle\sum_{j=0}^{N-1} \left(\boldsymbol{s}_j^\top Q \boldsymbol{s}_j + \boldsymbol{u}_j^\top R \boldsymbol{u}_j\right) + \boldsymbol{s}_N^\top P_N \boldsymbol{s}_N \\ \text{s.t.} \quad & \boldsymbol{s}_{j+1} = A_d \boldsymbol{s}_j + B_d \boldsymbol{u}_j + \hat{\boldsymbol{d}}, \quad f_{\min} - \bar{f}_i \le u_{j,i} \le f_{\max} - \bar{f}_i, \end{aligned} \tag{5}$$

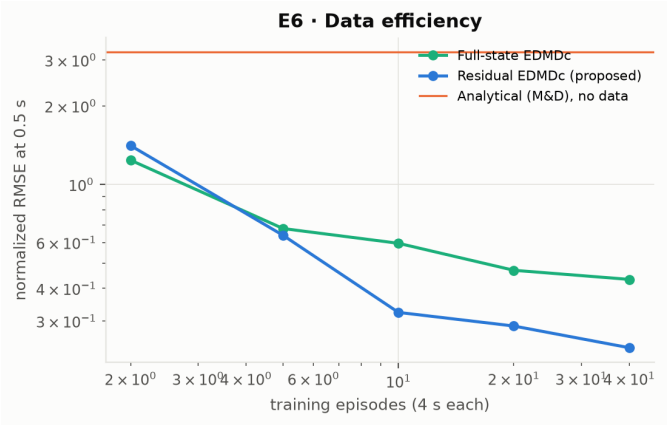


Figure 3: Prediction error at 0.5 s versus the number of training episodes (family A).

with OSQP [27] through CVXPY [28]. The analytical MPC uses $\hat{\boldsymbol{d}} = 0$. The residual-Koopman MPC evaluates the learned residual at the current state outside the optimization and holds it constant over the horizon, as in [10], so the problem stays a convex QP. Table 6 and Fig. 4 show the results. On family A, the thrust loss causes a steady altitude error of about 0.29 m for LQR and the analytical MPC. The residual-Koopman MPC reduces it to 0.19 m, a 32% reduction, similar to the 35% reported by Todde et al. for healthy flight. On family B all three controllers have small errors. Used as the capture controller after a detection delay (two-rotor spin-up, then the controller), neither MPC improved on LQR. The limiting factor there is the validity of the linear model far from the spin, not constraint handling.

Table 6: Altitude regulation under model mismatch (8 starts, 1 m below the set-point, 8 s).

Controller	Plant	Steady altitude error	Altitude RMS	QP time (median)
LQR	A	−0.285 m	0.397 m	—
MPC, analytical	A	−0.286 m	0.398 m	4.8 ms
MPC, residual Koopman	A	**−0.194** m	0.340 m	4.9 ms
LQR	B	−0.029 m	0.273 m	—
MPC, analytical	B	−0.033 m	0.275 m	5.1 ms
MPC, residual Koopman	B	+0.022 m	0.265 m	4.3 ms

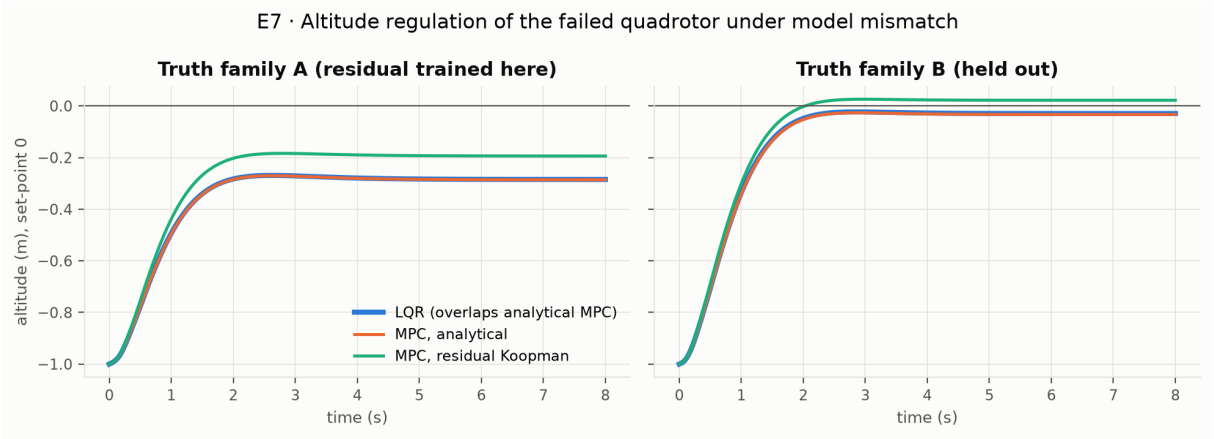


Figure 4: Altitude regulation of the failed vehicle under model mismatch.

7 RQ2: Fault-detection delay

7.1 The undetected phase

At $t = 0$ rotor 4 fails. Until the failure is detected at $t = \tau$, the healthy LQR keeps running and its command to rotor 4 is lost. Near hover the body rates are small, so the gyroscopic terms in (2) vanish and the roll equation reduces to $I^B_{xx}\dot{p} = l(f_2 - f_4) = lf_2$. The healthy controller reacts by reducing f_2, but a running motor cannot go below $f_{\min}$. Therefore

$$\dot{p} \geq \frac{lf_{\min}}{I^B_{xx}} = 10.6 \text{ rad/s}^2, \tag{6}$$

for any tuning of the healthy controller. Figure 5 confirms this: the tilt reaches about 13° at 100 ms and 37° at 200 ms, and three different tunings barely differ.

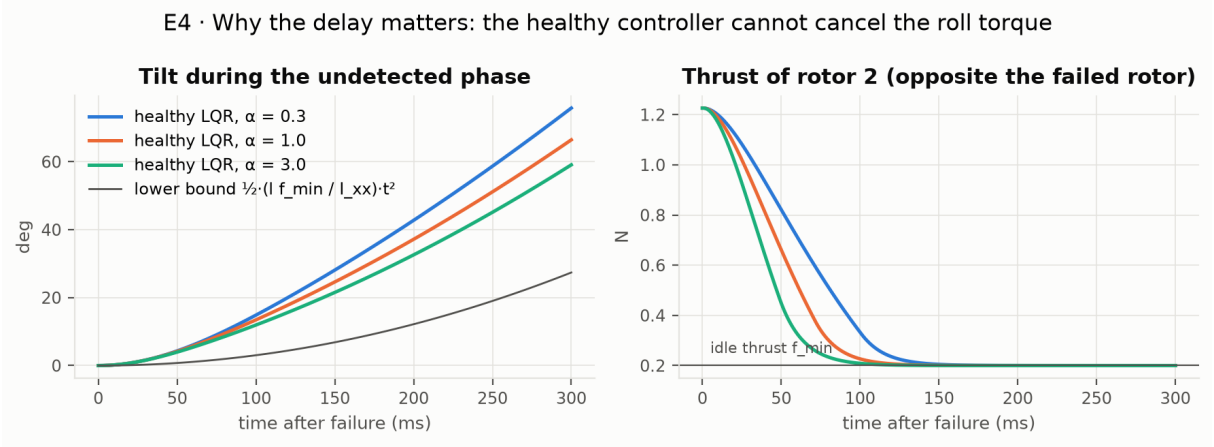


Figure 5: Undetected phase: tilt for three healthy-controller tunings (with the analytic lower bound) and the thrust of rotor 2, which falls to its idle limit.

7.2 Effect of detection delay

We simulated hover, rotor failure at $t = 0$, the healthy LQR until $t = \tau$, and then the strategy of [1]: spin up on rotors 1 and 3 until $\|\boldsymbol{\omega}\| > 10$ rad/s, then the relaxed-hover LQR. We fixed the success criterion in code before running the sweep. A run is *safe* if the attitude is captured ($\|\boldsymbol{n} - \bar{\boldsymbol{n}}\| < 0.05$ and $\|\boldsymbol{\omega} - \bar{\boldsymbol{\omega}}\| < 2$ rad/s over the last second), the altitude loss is below 2 m, and the vehicle never turns upside down. With instant detection the strategy loses 0.41 m of altitude, consistent with the paper's flight. With delay it degrades quickly (Table 7, Fig. 6). Reported detectors need 30–130 ms [17]. The tolerance did not change for three tunings of the healthy controller. With the looser criterion "attitude captured" the tolerance is about 40 ms. Switching the LQR on directly, without spin-up, captures the attitude but turns the vehicle upside down and loses 10–16 m.

Table 7: Spin-up + LQR strategy: safe recoveries over 20 random pre-failure states (velocity $\sigma = 0.3$ m/s, angles $\sigma = 3°$, rates $\sigma = 0.3$ rad/s).

Delay τ	0 ms	10 ms	20 ms	30 ms	50 ms	80 ms
Safe	20/20	17/20	6/20	1/20	0/20	0/20
Median altitude loss	0.38 m	1.11 m	3.20 m	9.12 m	39.1 m	76.5 m

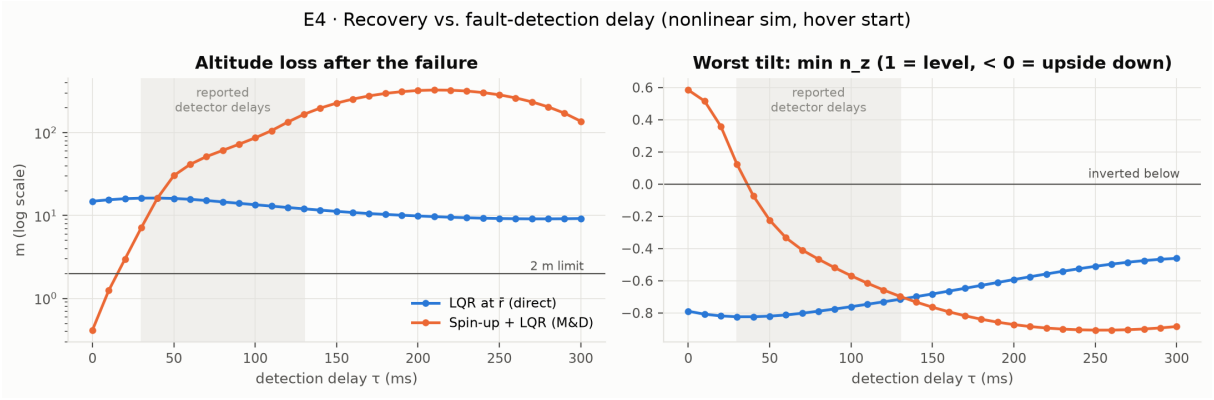


Figure 6: Altitude loss and worst tilt versus detection delay (nominal start). The shaded band marks reported detector delays.

7.3 A first saturation-aware certificate

For the closed loop $\dot{\boldsymbol{x}} = A\boldsymbol{x} + B\,\mathrm{sat}(F\boldsymbol{x})$ with $F = -K$ and symmetric saturation levels $\bar{\boldsymbol{u}}$, the ellipsoid $\mathcal{E} = \{\boldsymbol{x} : \boldsymbol{x}^\top Q^{-1}\boldsymbol{x} \le 1\}$ is contractively invariant if there is a matrix $Y = HQ$ such that [29, 30]

$$AQ + QA^\top + B(DFQ + D^-Y) + (DFQ + D^-Y)^\top B^\top + 2\alpha Q \preceq -\epsilon I \quad \forall D \in \mathcal{D}, \tag{7}$$

$$\begin{bmatrix} \bar{u}_i^2 & Y_i \\ Y_i^\top & Q \end{bmatrix} \succeq 0, \quad i = 1, \ldots, m, \tag{8}$$

where $\mathcal{D}$ is the set of diagonal 0/1 matrices, $D^- = I - D$, and α is a required decay rate. We maximize the extent of $\mathcal{E}$ along the direction from the equilibrium to the state right after a failure in hover. The motor states are bounded by their physical range; without this bound the problem is ill-conditioned and certifies a physically meaningless set. The certificate covers $\beta = 0.22$ of the way to the hover-onset state. In nonlinear simulation, every start up to $\beta = 0.7$ recovers without inverting, so the certificate is about three times conservative in this direction. On a two-dimensional slice (Fig. 7), the LQR fails only when the spin rate is close to zero, which is where the hover-onset state lies. A certificate built from the linearization at the full spin cannot reach that region. This is the motivation for the next step: a certificate that is valid over the whole spin-up, treating the yaw rate r as a varying parameter (LPV).

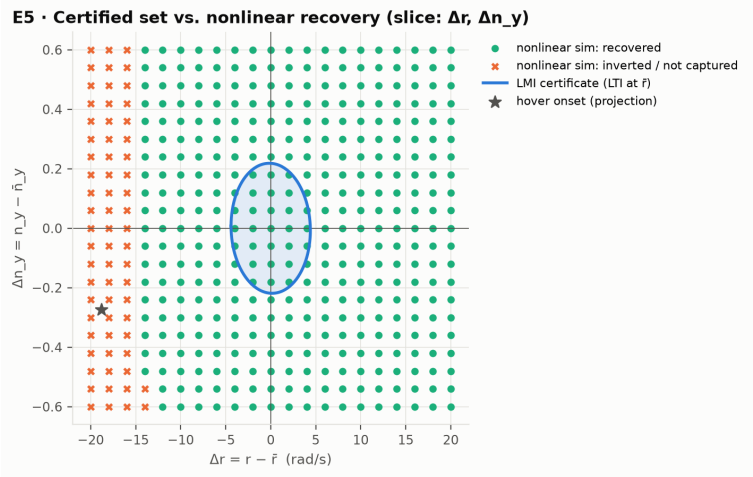


Figure 7: Slice $(\Delta r, \Delta n_y)$ of the certified ellipsoid (decay rate $0.1\,\mathrm{s}^{-1}$) against a grid of nonlinear simulations. 376 of 441 grid points recover without inverting; 7% of these lie inside the certified slice.

8 Limitations

1. **Model versus flight.** The nominal model, which is the paper's model, spins up about 30% more slowly than the real vehicle. The delay tolerance of Section 7 depends on this, so it holds for the published model; a sensitivity study is planned.
2. **Generalization of the learned models.** The residual model helps only on the plant family it was trained on.
3. **Linear certificates.** The certificate holds for the linearized saturated loop. The gap to the nonlinear system is measured by simulation, not proven.
4. **Truth-plant realism.** The truth plants add thrust loss, inertia error and motor lag, but not yet air-relative thrust or quadratic drag, both of which are validated in our implementation of the IJRR model.
5. **Novelty.** Two closely related papers [3, 4] are behind paywalls and have not yet been read in full.
6. **Comparability.** The delay sweep used the nominal plant, while the MPC capture test used truth family A, so their absolute numbers differ.

9 Plan for the remaining weeks

Weeks	RQ1: residual Koopman	RQ2: delay budget
1–2	MPC on the full-state Koopman model (the second controller of the proposal); integral-action MPC baseline	Read [3, 4]; LPV model in r; certificate over the spin-up
3–4	Online residual adaptation (recursive least squares)	Reachable set of the undetected phase; containment test; first certified $\tau^\star$
5–6	Air-relative thrust in the truth plant; residual trained on spin-up data	Trade study of $\tau^\star$, including yaw drag and thrust model; $\tau^\star$ per controller; bounded-residual certificate
7–8	Ablations; write-up	Simulation check of the certified $\tau^\star$; final report

The main risk is that a single quadratic Lyapunov function over the whole spin-up is infeasible or too conservative. The fallbacks are a parameter-dependent Lyapunov function, or a chain of nested ellipsoids along the spin-up.

10 Conclusion

We built and validated a simulator of a quadrotor with complete rotor loss and set up both research questions. For RQ1, the residual Koopman model is the most accurate predictor on the training plant, needs less data than a full-state Koopman model, and reduces a steady altitude error by about one third inside a convex MPC. It does not yet generalize to an unseen plant. For RQ2, the published recovery strategy tolerates only about 10 ms of detection delay in simulation, far less than reported detectors need. A first LMI certificate confirms that certificates built at the spinning equilibrium cannot cover the moment of failure. The next phase builds a certificate over the spin-up and the certified delay budget.

A Reproducibility

The code is a Python package (`rotorfail/`) with one script per experiment in `experiments/`; all random seeds are fixed. Commands:

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest -q                          # 45 tests
.venv/bin/python experiments/v0_validation_report.py # model vs. papers
.venv/bin/python experiments/e1_equilibrium.py       # ... up to e7
```

Detailed documentation is in `docs/`. Generated tables are in `results/tables/` and figures in `results/figures/`.

References

[1] Mark W. Mueller and Raffaello D'Andrea. Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 45–52, 2014.

[2] Mark W. Mueller and Raffaello D'Andrea. Relaxed hover solutions for multicopters: Application to algorithmic redundancy and novel vehicles. *The International Journal of Robotics Research*, 35(8):873–889, 2016. doi: 10.1177/0278364915596233.

[3] Alexander Lanzon, Alessandro Freddi, and Sauro Longhi. Flight control of a quadrotor vehicle subsequent to a rotor failure. *Journal of Guidance, Control, and Dynamics*, 37(2), 2014.

[4] Johannes Stephan, Lorenz Schmitt, and Walter Fichter. Linear parameter-varying control for quadrotors in case of complete actuator loss. *Journal of Guidance, Control, and Dynamics*, 41(10):2232–2246, 2018. doi: 10.2514/1.G003441.

[5] Sihao Sun, Xuerui Wang, Qiping Chu, and Coen de Visser. Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors. *IEEE Transactions on Robotics*, 2020. doi: 10.1109/TRO.2020.3010626. arXiv:2002.07837.

[6] Sihao Sun, Matthias Baert, Bram Strack van Schijndel, and Coen de Visser. Upset recovery control for quadrotors subjected to a complete rotor failure from large initial disturbances, 2020. arXiv:2002.09425.

[7] Fang Nan, Sihao Sun, Philipp Foehn, and Davide Scaramuzza. Nonlinear MPC for quadrotor fault-tolerant control. *IEEE Robotics and Automation Letters*, 2022. arXiv:2109.12886.

[8] Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, 2018.

[9] Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25:1307–1346, 2015.

[10] Edoardo Todde, Simone Martini, Alessandro Rizzo, and Kimon P. Valavanis. Residual Koopman-based model predictive control of quadrotors. In *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 50–56, 2026. doi: 10.1109/ICUAS69441.2026.11598621.

[11] D. Bruder, D. Bombara, and R. J. Wood. A Koopman-based residual modeling approach for the control of a soft robot arm. *The International Journal of Robotics Research*, 2025.

[12] Narayanan and Vaidya. RK-MPC: Residual Koopman model predictive control for quadruped locomotion in offroad environments, 2026. arXiv:2604.04221.

[13] Simha, Vadgama, and Raha. A geometric approach to rotor failure tolerant trajectory tracking control design for a quadrotor, 2017. arXiv:1704.00327.

[14] Mohamed Al Lawati and Adeel Akhtar. Quasi-static fault-tolerant feedback control of a quadrotor under rotor failure with provable safety guarantees, 2026. arXiv:2609.23918.

[15] Eeshan Kulkarni, Narasimhan Sundararajan, and Suresh Sundaram. Heli-quad design for full-attitude fault-tolerant control under complete failure of an actuator, 2024. arXiv:2011.07529v3.

[16] Alessandro Freddi, Alexander Lanzon, and Sauro Longhi. A feedback linearization approach to fault tolerance in quadrotor vehicles. In *Proc. 18th IFAC World Congress*, pages 5413–5418, 2011.

[17] IEEE2025. Failure detection and recovery for quadrotors in the presence of severe rotor failures with multiple model $\mathcal{L}_1$ adaptive controller. *IEEE Xplore document 11235993*, 2025. Author list to be verified.

[18] Florin Stoican and Sorin Olaru. *Set-Theoretic Fault-Tolerant Control in Multisensor Systems*. Wiley-ISTE, 2013.

[19] Jong-Yeob Shin and Christine M. Belcastro. Performance analysis on fault tolerant control system. *IEEE Transactions on Control Systems Technology*, 14(5):920–925, 2006.

[20] AST2024. Fault tolerant motion planning for a quadrotor subject to complete rotor failure. *Aerospace Science and Technology*, 2024. Author list to be verified.

[21] Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control. *PLoS ONE*, 11(2):e0150171, 2016.

[22] S. S. Narayanan, D. Tellez-Castro, S. Sutavani, and U. Vaidya. SE(3) Koopman-MPC: Data-driven learning and control of quadrotor UAVs. *IFAC-PapersOnLine*, 56(3):607–612, 2023.

[23] S. Martini, E. Todde, M. Stefanovic, M. J. Rutherford, A. Rizzo, and K. P. Valavanis. Koopman-based model predictive control of quadrotors. In *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 207–213, 2025.

[24] Lee et al. Koopman fault-tolerant model predictive control for multirotor with actuator fault, 2024. Cited in the project proposal; attribution to be verified.

[25] Lee et al. Data-driven fault detection and isolation for multirotor system using Koopman operator. *Journal of Intelligent & Robotic Systems*, 2024. Author list to be verified.

[26] Fu et al. Residual Koopman model predictive control for enhanced vehicle dynamics with small on-track data input, 2025. arXiv:2507.18396; author list to be verified.

[27] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.

[28] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

[29] Tingshu Hu and Zongli Lin. *Control Systems with Actuator Saturation: Analysis and Design*. Birkhäuser, Boston, 2001.

[30] Stephen Boyd, Laurent El Ghaoui, Eric Feron, and Venkataramanan Balakrishnan. *Linear Matrix Inequalities in System and Control Theory*. SIAM, Philadelphia, 1994.